

1. [a] The four principles of object-oriented programming are encapsulation, abstraction, inheritance and polymorphism. Write one program to incorporate all these features. Explain how the program makes use of each of them, with reference to specific lines in the code. [5]

[b] Explain the effect of the access modifiers public, protected, private, or default (no modifier) in Java. [2]

[c] Consider the statement "public static of a Java class should be final and static" [2]

(i) Explain why this is considered good programming practice. [2]

(ii) Give an example of a member variable that should not be declared public, even if final and static. [2]

2. JNU manages its students using a program that has a class Student with subclasses FirstYear, SecondYear, and ThirdYear for year-specific state and behaviour. The program has a List<Student> that contains all Students.

[i] Should Student be a class, an abstract class or an interface? Explain your answer in detail (pros and cons) [2]

[ii] Also at the end of the academic year, First-year and second-year students are promoted to the next academic year.

[i] Explain why this class design makes this problematic. [3]

[ii] Propose an alternative design and explain in detail how it addresses the problems you identified [3]

3. [a] Explain the meaning of the Liskov-Substitution principle. [2]

[b] Explain the meaning of the Single Responsibility principle. [2]

[c] Imagine that you are working in the insurance context. You can issue a Policy to insure a policy holder. There are two types of policies available: a life insurance policy and a car insurance policy. Both of these policies take into consideration the age of the policy holder. You need to calculate the premium (i.e. the cost) of each policy based on the following guideline:

- The life insurance takes 5 percent of the total sought coverage amount if the policy holder is under 35 of age or 10 percent otherwise
- For adults only, the car insurance takes 10 percent of the car value if the policy holder is over 30 of age or 20 percent otherwise

Your task is to implement a Java program using the following statement:

Define four classes PolicyHolder, Policy, LifeInsurancePolicy extends Policy, CarInsurancePolicy extends Policy such that they encapsulate this system and demonstrate a violation of the single Responsibility Principle in the Policy class and a Liskov Substitution Principle violation in the CarInsurance Policy class. [10]

4. You are designing a high-performance file processing application that reads multiple large files concurrently, processes their contents, and writes the results to a shared database. To ensure efficiency and proper resource management, multiple multithreading mechanisms in Java should be used. [3x7=21]

P T O

Your task is to implement a Java program using the following multithreading concepts:

1. **Thread creation** using both Thread and Runnable.
2. **Thread pooling** using ExecutorService.
3. **Synchronization** using synchronized and Lock (ReentrantLock).
4. **Inter-thread communication** using wait() and notify().
5. **Atomic operations** using AtomicInteger.
6. **Thread coordination** using CountdownLatch.
7. **Parallel execution** using ForkJoinPool.

- Create a system where multiple threads read different files concurrently.
- Each file-processing thread should extract words from the file and store them in a shared `HashMap<String, Integer>` (word-frequency counter).
- The program should ensure that multiple threads **safely update** the shared `HashMap`.
- After all files are processed, a single thread should write the final word count into the database.
- Use **inter-thread communication** to notify the main thread when all file-processing threads have finished.

5. You are developing an **online banking system** that allows users to perform transactions such as **withdrawals, deposits, fund transfers, and account balance checks**. The system must handle multiple types of errors gracefully and ensure that no transaction corrupts the system's state.

[3x7=21]

Your implementation should use the following exception handling techniques:

1. **Checked Exceptions (throws)** to indicate recoverable errors.
2. **Unchecked Exceptions (RuntimeException)** to handle programmer mistakes.
3. **Try-Catch-Finally** to handle critical operations safely.
4. **Multiple Catch Blocks** to manage different error types.
5. **Throwing Custom Exceptions** to define application-specific errors.
6. **Try-with-Resources** for safely handling resources like files or database connections.
7. **Nested Try-Catch Blocks** for handling multiple layers of failure.

MCA-II semester/2025
CS-410(Object Oriented Programming)

Time: 3 hours

Max. Marks: 60

1. You are designing a ticket booking system for a movie theatre where multiple users can book tickets simultaneously. Explain how you would handle concurrency issues in such a system using Java multithreading concepts. Discuss synchronization, deadlock possibilities, and real-life consequences of improper thread handling. [10]

2. You are designing a ride-sharing application (like Uber or Ola) where there are different types of vehicles: Car, Bike, and Auto. Each vehicle type has a different pricing strategy and seating capacity, but they all share some common behavior such as `'startRide()'`, `'endRide()'`, and `'calculateFare()'`.

How would you design this system using inheritance in Java? Explain how inheritance helps in code reuse, flexibility, and future scalability, and also mention any potential drawbacks in this design. [10]

3. You're building a banking application that processes transactions. Each transaction involves checking account balance, deducting money, and updating the database. However, during high traffic, issues like database failures, invalid inputs, and thread interruptions can occur. [10]

Design a robust exception-handling structure using Java to handle the following requirements:

1. Gracefully handle custom business exceptions like `'InsufficientFundsException'`.
2. Ensure that the database connection is always closed properly, even if an exception occurs.
3. Prevent the application from crashing unexpectedly, but still log the root cause for debugging.
4. Discuss how `'try-catch-finally'`, `'throws'`, and custom exceptions play a role in building this fault-tolerant system.

4. What is the difference between `'Thread'` class and `'Runnable'` interface in Java, and when should you prefer one over the other? [5]

5. What is the difference between "is-a" and "has-a" relationships in Java, and how are they implemented in code? Give examples for both. [5]

6. Write TRUE and FALSE for the following statements: [20]

- a. All methods in an abstract superclass must be declared abstract. ✗
- b. A class declared final cannot be subclassed. ✓
- c. Every method of a final class is implicitly final. ✗
- d. A redefinition of a superclass method in a subclass need not have the same signature as the superclass method. Such a redefinition is not method overriding but is simply an example of method overloading. ✓

- e. A constructor is a special method with the same name as the class that is used to initialise the members of a class object. Constructors are called when objects of their classes are instantiated. ✓
- f. A method declared static cannot access non-static class members. A static method has no this reference because static class variables and static methods exist independent of any objects of a class. ✓
- g. An array subscript may be an integer or an integer expression. If a program uses an expression as a subscript, then the expression is evaluated to determine the particular element of the array. ✓
- h. To pass one row of a double-subscripted array to a method that receives a single-subscripted array, simply pass the name of the array followed by the row subscript. ✓
- i. Overloaded methods can have different return values, and must have different parameter lists. Two methods differing only by return type will result in a compilation error. ✓
- j. Any block of Java code may contain variable declarations. ✓